NVIDIA: Getting Started with Deep Learning

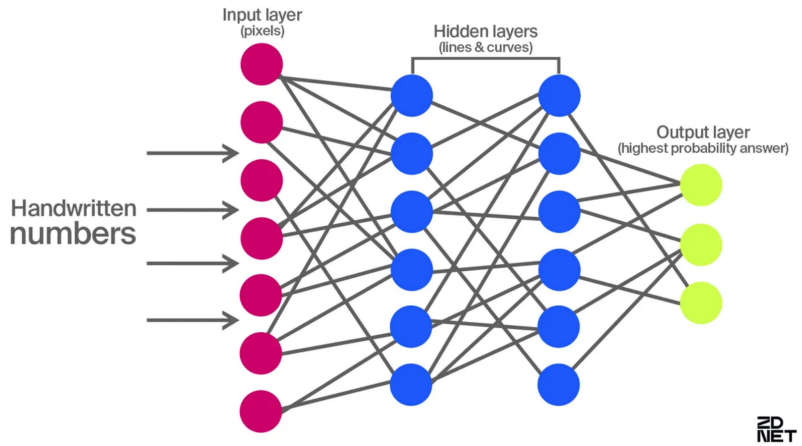
Course home page: https://learn.nvidia.com/courses/course-detail

Introduction to Deep Learning

Deep learning is a specialized subset of machine learning within artificial intelligence (AI) that uses multi-layered artificial neural networks, inspired by the human brain, to automatically learn complex patterns from vast amounts of data, enabling advanced tasks like image recognition, natural language processing (NLP), and self-driving cars with minimal human intervention.

The "deep" refers to the multiple interconnected layers of neurons that progressively extract features, allowing computers to understand complex information, as seen in voice assistants (Siri, Alexa) and generative AI.
How it works

- **Artificial Neural Networks (ANNs)**: These are computational structures with layers of interconnected nodes (neurons).
- **Layered Processing:** Data passes through an input layer, several hidden layers, and an output layer. Each layer learns to recognize increasingly complex features from the data.
- **Feature Learning**: Unlike traditional machine learning, deep learning models learn relevant features automatically from raw data (like pixels in an image or words in text), rather than needing humans to define them.
- **Training**: Models are trained on massive datasets, adjusting connections between neurons (weights and biases) to improve accuracy, similar to how the brain adapts.



Key characteristics

- Deep learning is a more advanced form of machine learning.
- Requires huge volumes of data to train effectively.
- Can self-correct and find patterns without explicit programming for every scenario.

Foundational ANN Concepts

Tensors

A scalar value has 1 dimension, its magnitude. Examples would be **temperature** or the value of **pi**
A vector has 2 dimensions: direction and magnitude. Examples would be **velocity**, **gravity**
A tensor is an array with n dimensions.

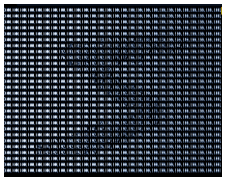
Example 1: Imagine there are many encyclopedia books, with many chapters, each chapter having many pages and each page having many lines. Therefore, the shape of that tensor will be something like this:

```
tensor[7, 5, 3, 2]
        │  │  │  └─ Lines #2 (innermost/deepest)
        │  │  └──── Pages #3 (nested in chapter)
        │  └─────── Chapter #5 (nested in book)
        └────────── Book #7 (outermost)
```

Example 2: When a greyscale image is converted to a tensor, the dimension is represented like (1, 28, 28), 1 representing the grey channel, 28 x 28 representing the width and height of 28 pixels. Tensor visualized from the first lab (1 x 28 x 28):

```
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0118, 0.0706, 0.0706, 0.0706, 0.4941, 0.5333, 0.6863, 0.1020, 0.6510, 1.0000, 0.9686, 0.4980, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1176, 0.1412, 0.3686, 0.6039, 0.6667, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.8824, 0.6745, 0.9922, 0.9490, 0.7647, 0.2510, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1922, 0.9333, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.9843, 0.3647, 0.3216, 0.3216, 0.2196, 0.1529, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0706, 0.8588, 0.9922, 0.9922, 0.9922, 0.9922, 0.9922, 0.7765, 0.7137, 0.9686, 0.9451, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3137, 0.6118, 0.4196, 0.9922, 0.9922, 0.8039, 0.0431, 0.0000, 0.1686, 0.6039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0549, 0.0039, 0.6039, 0.9922, 0.3529, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.5451, 0.9922, 0.7451, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0431, 0.7451, 0.9922, 0.2745, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1373, 0.9451, 0.8824, 0.6275, 0.4235, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3176, 0.9412, 0.9922, 0.9922, 0.4667, 0.0980, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1765, 0.7294, 0.9922, 0.9922, 0.5882, 0.1059, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0627, 0.3647, 0.9882, 0.9922, 0.7333, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9765, 0.9922, 0.9765, 0.2510, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1804, 0.5098, 0.7176, 0.9922, 0.9922, 0.8118, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1529, 0.5804, 0.8980, 0.9922, 0.9922, 0.9922, 0.9804, 0.7137, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0941, 0.4471, 0.8667, 0.9922, 0.9922, 0.9922, 0.9922, 0.7882, 0.3059, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0902, 0.2588, 0.8353, 0.9922, 0.9922, 0.9922, 0.9922, 0.7765, 0.3176, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0706, 0.6706, 0.8588, 0.9922, 0.9922, 0.9922, 0.9922, 0.7647, 0.3137, 0.0353, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.2157, 0.6745, 0.8863, 0.9922, 0.9922, 0.9922, 0.9922, 0.9569, 0.5216, 0.0431, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.5333, 0.9922, 0.9922, 0.9922, 0.8314, 0.5294, 0.5176, 0.0627, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000]
```

Each floating point value represents whether is not lit up, represented by 0 and if a pixel is lit up, the with what intensity it is lit up, values closer to one is more bright, values closer to 0, less bright. This is how CPU/GPU understands an image data.



If I shrink the screenshot above and increase the contrast, you can visually start to see that the numerical data has a pattern, like a number.

Here is the original image from MNIST dataset that was converted into a tensor above

Deep dive:

- Why use floating point:
    a. Architecturally, GPUs and TPUs are designed to process massive floating point parallelly.
    b. Mathematically, applying gradient descent on whole numbers can be catastrophic since the adjustments may exponentially explode over the number of layers in backpropagation; however, making fractional updates to the weights and biases gives an opportunity to self-correct.
- Why is pixel color or intensity data represented from 0 to 255:
    a. Computers use bits; an 8-bit represents a number from 0 to 255 in binary. Example:
        - 00000000 = 0
        - 00000001 = 1
        - 00000010 = 2
        - 00000011 = 3
    b. Thus allowing to store the information with smallest value possible. We can assign the intensity value of a pixel using 256 allocated values. If we multiply 256 pixel intensity x 3 color channels (RGB) we get 16.77 million distinct values, this is a decent number of colors represented in the real world.

Epoch

One epoch = one complete pass through your **entire training dataset**.

If you have 10,000 images, one epoch means the model has seen all 10,000 once. Training usually requires multiple epochs (10, 50, 100+) because the model needs repeated exposure to learn the patterns well.

Batch Size

Instead of updating weights after every single epoch (slow) or after the entire dataset (memory-intensive), you process data in **batches** — chunks of examples at a time.

Common batch sizes: 32, 64, 128, 256

Benefits:

- The biggest advantage is progressive backpropagation, because by the time you are done with 1st epoch, assuming you have several batches, you would have tweaked your learning rate to effectively use the next epoch and it's batches.
- More stable gradient estimates than single examples
- Fits in GPU memory
- Faster than processing one at a time

Step

A step (or iteration) is **one weight update** — processing one batch and adjusting the parameters.

Say you have 10,000 training images and a batch size of 100:

- 1 step = process 100 images, update weights once
- 1 epoch = 100 steps (10,000 ÷ 100)
- 10 epochs = 1,000 total steps

Each step, gradient descent uses the learning rate to decide how much to adjust m and b based on the gradient, slowly minimizing MSE across all your data.

Calculating Accuracy

- Human-readable - "92% accurate" is easy to understand vs. "loss = 0.342"
- Track progress - See if the model improves each epoch
- Know when to stop - If accuracy plateaus, training more won't help
- Detect problems - Low accuracy (like 16% for 6 classes) = model isn't learning

Loss vs. Accuracy

- **Loss**: Math metric for training (how wrong the predictions are)
- **Accuracy**: Human metric for evaluation (% of correct predictions)

Flattening the image dimensions

We only flatten for simple fully connected networks.

For CNNs (Convolutional Neural Networks), we DON'T flatten initially because:

- CNNs preserve spatial relationships (neighboring pixels matter)
- Convolutions work on 2D data directly

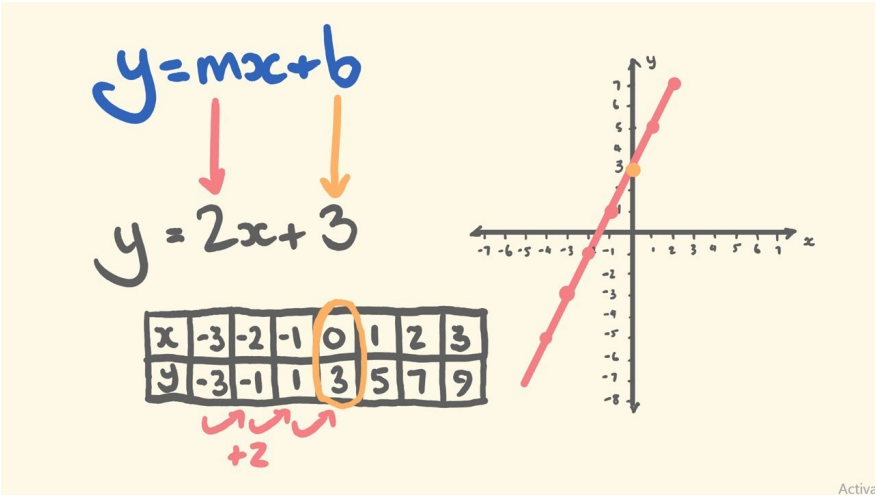
before flattening
[32, 1, 28, 28]
↑ └──┬──┘
batch | image dimensions

after flattening
[32, 784]
↑ ↑
batch | all image pixels combined (1×28×28 = 784)

Introductory Mathematical Concepts in Machine Learning

Slope-intercept

Y = mx + b



This is the **slope-intercept form** of a linear equation, the fundamental equation for a straight line.

Breaking it down:

- **Y** = the output (dependent variable)
- **m** = the slope (how steep the line is)
- **x** = the input (independent variable)
- **b** = the y-intercept (where the line crosses the y-axis when x=0)

The slope (m) tells you how much Y changes for every 1-unit increase in x. A slope of 2 means Y goes up 2 for every 1 step in x. A negative slope means the line goes downward.

The y-intercept (b) is your starting point — the value of Y when x is zero.

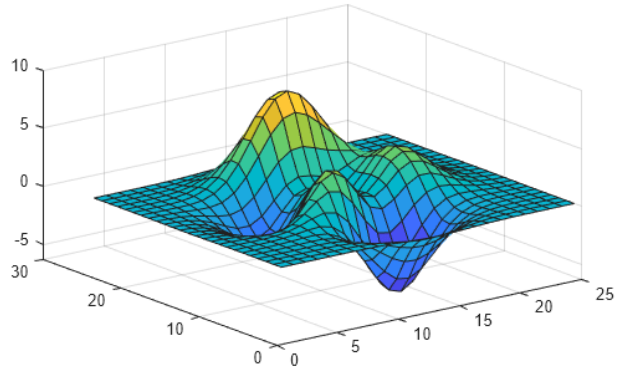
Why it matters for ML:

This is essentially what a single neuron or linear regression does. In machine learning terms:

- **x** = your input features
- **m** = the weights (what the model learns)
- **b** = the bias term
- **Y** = the prediction

When you're training a neural network, you're essentially trying to find the optimal values of m and b that minimize the error between your predictions and the actual values. The MNIST dataset uses this concept at its core — each neuron computes a weighted sum of inputs plus a bias, which is just a multidimensional version of y = mx + b.

Surface plot



It is a 3D visualization that shows how an output (Z) changes based on two inputs (X and Y).

Think of it as a topographical map brought to life — hills, valleys, and slopes rendered in 3D.

In the context of linear regression / ML:

When you're training a model with y = mx + b, you're trying to find the best values for m (slope) and b (bias). A surface plot can visualize the **loss landscape**:

- **X-axis** = possible values of m
- **Y-axis** = possible values of b
- **Z-axis (height)** = the MSE/error for that combination of m and b

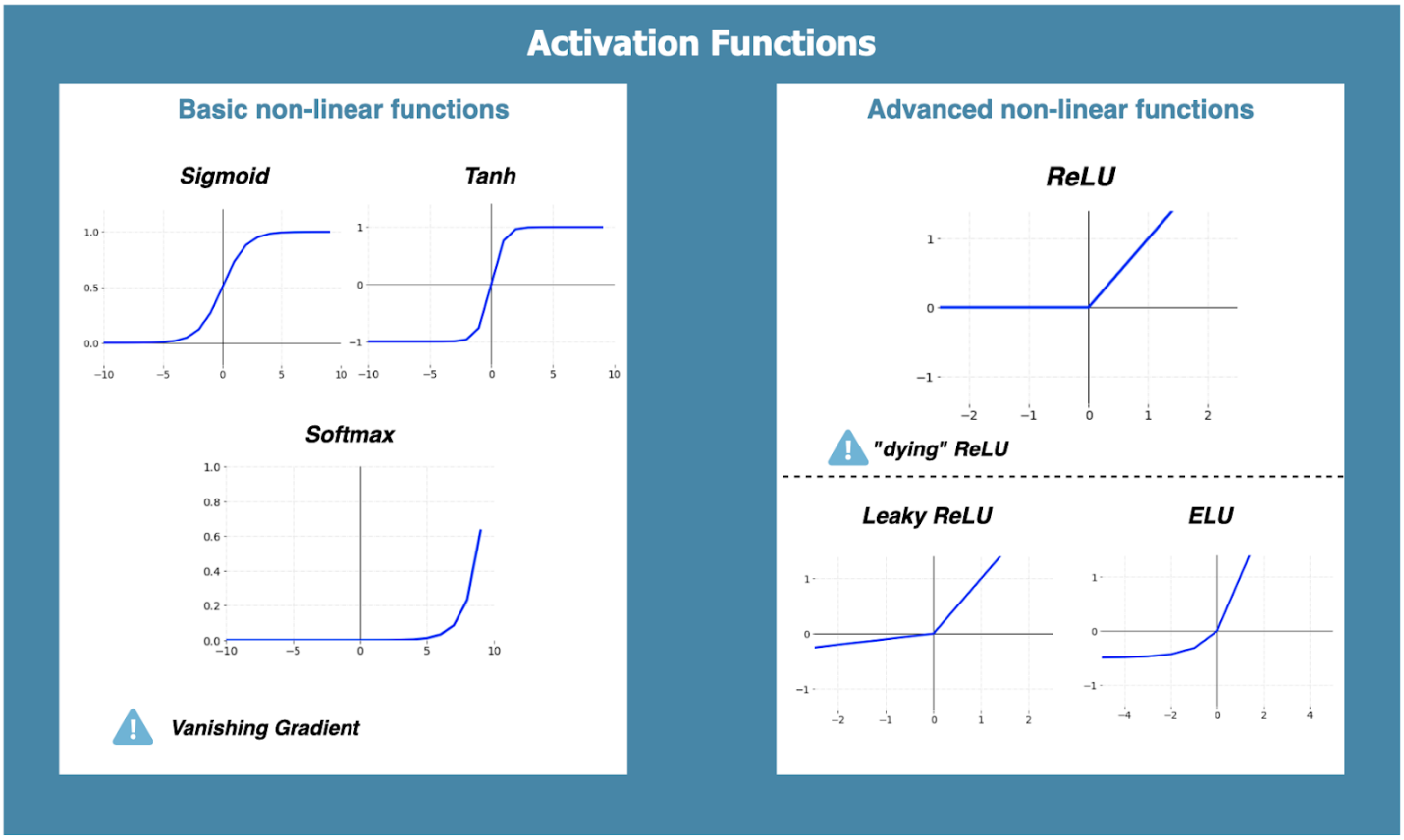
The surface shows you a "bowl" shape where:

- High points = bad combinations (high error)
- The lowest point = optimal values of m and b (minimum error)

Activation Functions

Activation functions decide whether a neuron should "fire" (activate) and transform its output in a non-linear way. Without activation functions, stacking multiple layers would just collapse into one big linear equation — no matter how deep your network, it could only learn straight lines.

Activation functions adds non-linearity, allowing networks to learn complex patterns like curves, edges, and abstract features.



ReLU (Rectified Linear Unit)

If input > 0, output = input. Otherwise, output = 0. Simple, fast, most widely used. Keeps positive values, kills negatives.

Sigmoid

Squashes output between 0 and 1. Good for probabilities. Used in the output layer for binary classification.

Tanh

Squashes output between -1 and 1. Like sigmoid but centered at zero.

Softmax

converts a vector of numbers into probabilities that sum to 1. Used in the output layer for multi-class classification (like MNIST — "30% chance it's a 3, 65% chance it's an 8...")

Loss Function

A loss function measures how wrong your model's predictions are compared to the actual values.

The objective is trying to minimize the loss. Loss functions can be broadly grouped into two categories:

1. **Regression**: predicts continuous values like price, temperature, sales, etc.
    a. MSE: penalizes larger errors more heavily due to squaring
    b. MAE: less sensitive to outliers than MSE
    c. RMSE: Reverses the value to the original value from MSE
    d. Huber:
2. **Classification**: predicts categories or labels like spam/not spam, disease/no disease, etc.
    a. Cross-entropy: Adjusts the loss depending on the size of the error
    b. BCE With Logits Loss: binary or multilabel, yes/no on each class independently

The loss function gives the optimizer a score to improve upon. Lower loss = better model.

MSE (Mean Squared Error)

Once you have your line Y = mx + b, making predictions, you need to know how far off those predictions are from the actual values.

MSE = average of (actual - predicted)^2

For each data point, you calculate the error (how far off you were), square it, and then average all those squared errors together.

Why square it?

- Eliminates negative values (being off by -5 is just as bad as +5)
- Penalizes large errors more heavily (an error of 10 becomes 100, while an error of 2 becomes just 4)

RMSE (Root Mean Squared Error)

RMSE = √MSE, you just take the square root of MSE. This brings the error back into the **same units** as your original data, making it more interpretable.

If you're predicting house prices in dollars, MSE gives you "squared dollars" (meaningless), while RMSE gives you actual dollars.

How they connect to y = mx + b:

During training, the model adjusts m and b to **minimize** MSE (or RMSE). This is the optimization process — gradient descent nudges the weights little by little until the error is as small as possible. That's essentially what "learning" means in machine learning.

Huber

Huber loss behaves like two different functions depending on how big the error is, controlled by a threshold parameter called **delta (δ)**:

- **When the error is small** (≤ δ) → it acts like MSE (squared error), giving you smooth gradients near zero for precise convergence
- **When the error is large** (> δ) → it switches to MAE (linear error), growing linearly instead of quadratically

This piecewise design solves a specific problem. If you have an outlier where your prediction is off by 1000, MSE contributes 1,000,000 to the loss. That single outlier can dominate the entire gradient and yank your model's weights in a bad direction. Huber loss caps that influence by going linear beyond the delta threshold — the outlier still contributes to the loss, but proportionally rather than explosively.

Advantages over MSE:

- Robust to outliers — large errors don't dominate training
- Still differentiable everywhere (unlike MAE which has that sharp corner at zero)
- Smooth gradients near zero for stable, precise convergence in the final stages of training

Advantages over MAE:

- Better convergence near the optimum because of the quadratic region
- No undefined gradient at exactly zero

Common use cases:

- **Regression with noisy data** — real-world datasets often have outliers from sensor errors, data entry mistakes, or genuine anomalies. Financial data is a great example — stock returns have fat tails and occasional extreme values that would wreak havoc on MSE
- **Object detection** — this is actually where Huber loss became famous. Models like Faster R-CNN use it (as smooth L1 loss) for bounding box regression because predicted boxes can be wildly off early in training, and you don't want those huge initial errors destabilizing the gradient
- **Reinforcement learning** — DQN and related algorithms use Huber loss for the value function because reward signals can be noisy and spiky

Cross Entropy

Cross-entropy loss measures **how far your model's predicted probability distribution is from the true distribution**. In simple terms — if the correct answer is "cat" and your model says 95% cat, the loss is small. If it says 20% cat, the loss is large. It penalizes confident wrong answers very heavily because of the logarithm at its core.

The math in plain English: take the log of the probability your model assigned to the *correct* class, flip the sign. That's it. The model gets rewarded for putting high probability on the right answer and severely punished for putting low probability on it.

Two flavors in PyTorch:

- `nn.CrossEntropyLoss()` — multiclass, one correct answer out of N classes (cat vs dog vs bird)
- `nn.BCEWithLogitsLoss()` — binary or multilabel, yes/no on each class independently (an image can be both "outdoor" and "sunny")

Best use cases:

- **Classification** — this is its home territory. Image classification, text categorization, sentiment analysis, any problem where inputs map to discrete categories
- **Language models** — predicting the next token is fundamentally a classification over the vocabulary, so every GPT-style model trains with cross-entropy
- **Well-balanced datasets** — it works best when your classes are roughly equally represented. Once you hit severe imbalance, that's when you start looking at focal loss or class weighting
- **When you need calibrated probabilities** — cross-entropy naturally pushes the model toward outputting meaningful probability estimates, unlike hinge loss which only cares about the decision boundary

Other Loss Functions

Focal Loss is probably the most important alternative. It was introduced by Facebook AI for object detection (the RetinaNet paper). The core problem it solves is **class imbalance**. In standard cross-entropy, easy/well-classified examples still contribute to the loss and can overwhelm the signal from hard, rare examples. Focal loss adds a modulating factor that *down-weights easy examples automatically*. So if your model is already 99% confident about a sample, its contribution to the loss nearly vanishes, letting the model focus on the difficult cases. This is huge in scenarios like fraud detection, medical diagnosis, or any problem where one class massively outnumbers the other.

Hinge Loss comes from the SVM (Support Vector Machine) world. Instead of pushing predictions toward probability distributions, it only cares about whether the correct class scores higher than the wrong class by a sufficient *margin*. Once a sample is correctly classified by enough margin, its loss goes to zero — the model stops wasting effort on things it already gets right. Used in tasks where you care more about decision boundaries than calibrated probabilities.

Label Smoothing isn't a different loss function per se — it modifies cross-entropy by softening the target labels. Instead of telling the model "this is 100% cat, 0% everything else," you say "this is 90% cat, and spread the remaining 10% across other classes." This prevents the model from becoming overconfident, acts as a regularizer, and often improves generalization. It's widely used in modern architectures — the original Transformer paper used it.

Dice Loss / IoU Loss is common in image segmentation. Cross-entropy evaluates each pixel independently, which creates class imbalance problems when the thing you're segmenting is small relative to the background. Dice loss directly optimizes the overlap between predicted and ground truth regions, so it naturally handles that imbalance.

KL Divergence (Kullback-Leibler) measures how one probability distribution diverges from another. It's closely related to cross-entropy mathematically but is used differently — most commonly in variational autoencoders (VAEs) where you want to regularize a learned distribution to stay close to a prior (usually a standard normal). Also used in knowledge distillation, where you train a small model to mimic a larger model's output distribution.

Contrastive Loss / Triplet Loss takes a completely different approach — instead of classifying into fixed categories, it learns an *embedding space* where similar items are pulled close together and dissimilar items are pushed apart. This is how face recognition systems like FaceNet work, and it's the foundation of modern retrieval systems. You don't need predefined classes at all.

When to consider switching from cross-entropy:

- Severe class imbalance → Focal Loss
- You need calibrated confidence to not be overconfident → Label Smoothing
- Pixel-level segmentation with small targets → Dice Loss
- Learning similarity rather than fixed categories → Contrastive/Triplet Loss
- You want margin-based decisions, not probabilities → Hinge Loss

Gradients and Gradient Descent

The **gradient** is the slope of the surface at any given point — it tells you which direction is "uphill" and how steep it is. Mathematically, it's the derivative (rate of change) of the error with respect to your parameters.

Gradient Descent is an optimization algorithm used to find the minimum of a function. It's like searching for the lowest point (valley) in a hilly landscape. The idea is to take small steps in the steepest downhill direction until you reach the lowest point (or a point that's "good enough").

In machine learning, Gradient Descent is often used to minimize a "cost function." This cost function measures how far off your model's predictions are from the actual values. The goal is to adjust the model's parameters (like weights and biases) to minimize this cost, making your model as accurate as possible.

It works by:

- Calculate the gradient at your current position
- Take a step in the **opposite** direction (downhill)
- Repeat until you reach the bottom of the bowl

Think of it as being blindfolded on a hill — you feel which way is steepest, then step the other way. Keep doing that and you'll eventually reach the valley.

Learning Rate

This controls **how big each step is** during gradient descent.

- **Too high** = you overshoot the minimum, bouncing around or diverging entirely
- **Too low** = you inch along painfully slowly, taking forever to converge
- **Just right** = you descend efficiently to the minimum

It's typically a small value like 0.01 or 0.001. Finding the right learning rate is part art, part experimentation.

Optimizers

Optimizers are algorithms that adjust the model's weights during training to minimize the loss function as efficiently as possible.

They determine *how* the model learns — specifically, how fast and in what manner the weights get updated after each step based on the gradient.

SGD (Stochastic Gradient Descent)

Updates weights using the gradient from a random batch. Simple but can be slow and get stuck. Often paired with "momentum" — like a ball rolling downhill that builds up speed.

Adagrad

Adapts the learning rate per parameter. Parameters that update frequently get lower rates; rare ones get larger rates. Good for sparse data, but the learning rate can shrink too aggressively over time.

RMSprop

fixes Adagrad's shrinking problem by using a moving average of recent gradients instead of accumulating all history. Keeps learning rates from decaying to zero.

Adam (Adaptive Moment Estimation)

is the most popular. Combines the best of momentum + RMSprop. Tracks both the direction (momentum) and the magnitude (adaptive rates) of gradients. Works well out of the box for most problems.

Why Frameworks Use Them

- Less hyperparameter tuning — you don't have to manually schedule learning rate changes
- Faster convergence — they navigate the loss surface more efficiently
- More robust — they handle different types of problems without as much babysitting

Adam is the default choice for most practitioners. If it doesn't work well, people often try SGD with momentum as an alternative.

Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are neural networks designed specifically for processing grid-like data, especially images.

The Problem They Solve

A regular neural network looking at a 28x28 image (like MNIST) would flatten it into 784 individual pixels and lose all spatial information. It wouldn't understand that certain pixels are neighbors, or that an edge in the top-left is the same pattern as an edge in the bottom-right.

How CNNs Work

Instead of looking at every pixel independently, CNNs slide small **filters** (like 3x3 grids) across the image. Each filter learns to detect a specific feature.

Convolutional Layer Filters scan across the image, producing "feature maps." Early layers detect simple features (edges, corners). Deeper layers combine those into complex patterns (shapes, textures, eventually whole objects).

Pooling Layer Shrinks the feature maps by taking the max or average of small regions. Reduces computation and makes the network less sensitive to small shifts in position.

Fully Connected Layer After convolutions extract features, these layers at the end make the final classification decision.

Why They're Powerful

- **Parameter sharing** — the same filter is reused across the whole image, so fewer weights to learn
- **Translation invariance** — a cat is a cat whether it's in the corner or center
- **Hierarchical learning** — edges → shapes → parts → objects

CNNs power image classification, face recognition, medical imaging, self-driving cars — basically anything involving visual data.

Kernels

(also called filters) are the small matrices that slide across an image to detect features in a CNN.

A typical kernel is a 3x3 or 5x5 grid of learnable weights. During training, the network figures out what values these weights should have to detect useful patterns.

How They Work

The kernel moves across the image, and at each position it performs element-wise multiplication with the pixels underneath, then sums the result into a single value. This produces a new "feature map" showing where that pattern was detected.

What They Detect

Different kernels learn to detect different things:

- One kernel might detect vertical edges
- Another detects horizontal edges
- Another detects corners
- Deeper in the network, kernels detect complex patterns like eyes, wheels, textures

Example

A simple edge-detection kernel might look like:

```
[-1  0  1]
[-1  0  1]
[-1  0  1]
```

When this slides over an image, it produces high values where there's a vertical edge (sharp brightness change) and low values in flat regions.

Key Point

You don't hand-design these kernels — the network **learns** them through backpropagation. You just specify how many kernels per layer and their size. The training process discovers what features matter for your task.

Stride

is how many pixels the kernel moves each time it slides across the image.

Stride of 1 The kernel shifts one pixel at a time. You get a detailed, larger output — captures everything but slower to compute.

Stride of 2 The kernel jumps two pixels at a time. The output is roughly half the size in each dimension — faster, but skips over some positions.

Why It Matters

- **Stride 1** = more overlap, more detail, larger feature maps
- **Higher stride** = less overlap, downsamples the image, reduces computation

It's another way to shrink spatial dimensions (like pooling), and choosing the right stride affects both the output size and how much information is preserved.

Padding

is adding extra pixels (usually zeros) around the border of an image before applying the kernel.

The Problem

When a kernel slides across an image, it can't center on edge pixels — there's nothing outside the boundary. So the output shrinks. A 28x28 image with a 3x3 kernel becomes 26x26.

Stack several convolutional layers and your image keeps shrinking, losing spatial information at the edges.

The Solution

Add a border of zeros (or other values) around the image so the kernel can process edge pixels properly.

Common Types

"Valid" (no padding) No padding added. Output shrinks. Edge pixels get less attention.

"Same" padding Enough padding added so output size equals input size. A 28x28 stays 28x28.

Why It Matters

- Preserves spatial dimensions through multiple layers
- Ensures edge/corner information isn't lost
- Gives you control over output size

Pooling

Pooling is a downsampling operation that shrinks feature maps by keeping only the average or maximum value in each region.

How It Works

A small window (typically 2x2) slides across the feature map. For each position, it outputs only the highest value in that window.

Example with 2x2 max pooling:

[1 3]

[2 7] → 7

A 4x4 feature map becomes 2x2.

Why Use It

- **Reduces size** — fewer parameters, less computation
- **Keeps the strongest features** — the max value represents where the kernel detected the pattern most strongly
- **Adds translation invariance** — if a feature shifts slightly, the max in that region is likely the same

Alternative: Average Pooling

Takes the average instead of the max. Smoother but less common — max pooling tends to work better for preserving important features.

Dropout

It is a regularization technique that randomly "turns off" neurons during training to prevent overfitting.

How It Works

During each training step, a percentage of neurons (say 20% or 50%) are randomly ignored — their outputs set to zero. The network must learn to not rely too heavily on any single neuron.

Why It Helps

- **Prevents overfitting** — stops the network from memorizing training data
- **Forces redundancy** — multiple neurons learn similar features, making the network more robust
- **Acts like an ensemble** — each training step uses a slightly different architecture, like training many networks and averaging them

During Inference

Dropout is turned off when making predictions. All neurons are active, but their outputs are scaled to account for the difference.

Analogy

Imagine a group project where random team members call in sick each day. The team learns to share knowledge so no single person is critical. Result: a more resilient team that doesn't fall apart if someone is absent.

Intermediate ANN Concepts

Vanishing Gradients

During backpropagation, gradients get multiplied through each layer as they flow backward. With activation functions like sigmoid or tanh, those multiplied values can shrink exponentially, so by the time gradients reach the earlier layers, they're practically zero. Those layers barely update, effectively stopping them from learning.

This is one of the key reasons ReLU became so popular — it doesn't squash gradients the same way. And architectures like ResNets with skip connections tackle it by giving gradients a shortcut path back through the network. Worth noting that the first option (gradients becoming extremely large) describes the opposite problem — the **exploding** gradient problem.

Exploding gradients

are about gradients growing uncontrollably large — that's the mirror of the vanishing gradient problem you nailed earlier. But increasing the learning rate would actually make it worse, amplifying those already massive updates.

The correct answer is **gradient clipping**. It works by capping the gradient values to a maximum threshold before the weight update happens. If the gradient norm exceeds that threshold, it gets scaled down proportionally. Simple but effective — it puts a ceiling on how dramatic any single update can be, preventing weights from blowing up to infinity.

Other things that help with exploding gradients include proper weight initialization (like Xavier or He initialization), batch normalization, and using architectures with gated mechanisms like LSTMs.

Slow Convergence

Convergence is when the model's loss stops meaningfully decreasing — it's found a good solution. Slow convergence means the model is learning, but painfully slowly. It's taking way too many epochs to get to a good loss value.

Common causes include a learning rate that's too low (tiny steps toward the minimum), poor weight initialization (starting in a bad spot in the loss landscape), unnormalized input data, or using a basic optimizer like vanilla SGD without momentum. Think of it like navigating to a destination by taking one-inch steps — you'll get there eventually, but it's wildly inefficient. This is one reason Adam is popular — its adaptive per-parameter learning rates help avoid this problem. Batch normalization also helps by smoothing the loss landscape so gradient descent can take more direct paths.

Class Imbalance

This happens when training data has vastly unequal representation across classes. For example, if you're training a fraud detection model and 99% of transactions are legitimate and 1% are fraud, the model can achieve 99% accuracy by simply predicting "not fraud" every time — and learn nothing useful about detecting actual fraud.

Strategies to handle it include oversampling the minority class (duplicating or synthesizing examples using techniques like SMOTE), undersampling the majority class, adjusting class weights in the loss function so misclassifying the rare class is penalized more heavily, or using metrics beyond accuracy — like precision, recall, F1 score, or AUC-ROC — that actually reveal how well the model handles the minority class. This is a very real-world problem, especially in financial services where the events you care about most (fraud, defaults, anomalies) are often the rarest.

Batch normalization

normalizes the activations within each mini-batch to have roughly zero mean and unit variance before passing them to the next layer. This combats what's called internal covariate shift — where the distribution of inputs to each layer keeps changing as the weights update, making training unstable. The result is faster convergence, the ability to use higher learning rates, and it also provides a slight regularization effect.

Logits

These are the raw, unnormalized output scores from the final layer of a neural network before any probability transformation is applied. They can be any real number, positive, negative, large, or small.

To turn logits into probabilities, you pass them through a softmax function (for multi-class classification) or a sigmoid function (for binary classification). For example, if a 3-class classifier outputs logits of [2.0, 1.0, 0.1], softmax converts those into something like [0.59, 0.24, 0.17] — proper probabilities that sum to 1.0.

So the flow is: final layer outputs **logits** → scaled by **temperature** → passed through **softmax** → you get **probabilities** → the model samples from those probabilities to pick the next token.

Appendix

- **Foundational Mathematical Concepts**
  - Classification vs. Regression: https://www.geeksforgeeks.org/machine-learning/ml-classification-vs-regression/
  - Gradient Descent: https://www.linkedin.com/pulse/understanding-gradient-descent-python-rany-elhousieny-phd%E1%B4%AC%E1%B4%AE%E1%B4%B0/
  - What is convolution (3 blue 1 brown): https://www.youtube.com/watch?v=KuXjwB4LzSA
  -
- **Deep Learning Theory**
  - MIT Deep Learning Lecture Notes: https://ocw.mit.edu/courses/15-773-hands-on-deep-learning-spring-2024/lists/lecture-notes/
  - Michael Nielson's book on Deep Learning: http://neuralnetworksanddeeplearning.com/chap1.html
  - Deep Learning (by Ian Goodfellow et el): https://aikosh.indiaai.gov.in/static/Deep+Learning+Ian+Goodfellow.pdf
  - Google Machine Learning courses: https://developers.google.com/machine-learning
  - CNN Visual Explainer: https://poloclub.github.io/cnn-explainer/
  - Practice by solving DL exercises: https://www.deep-ml.com/problems
  -
- **Transformer Theory**
  - Transformer Visually Explained: https://poloclub.github.io/transformer-explainer/
  -
- **Sample Models**
  - TensorFlow Hub: https://www.tensorflow.org/hub
  - NVIDIA Catalog: https://catalog.ngc.nvidia.com/search?filters=resourceType|Model|model
  - HuggingFace:
  -